

Setup

```
# importing libraries
from pathlib import Path
import sys

import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder,
    StandardScaler,
    RobustScaler,
    FunctionTransformer
)
from sklearn.feature_selection import mutual_info_classif

import networkx as nx

from scipy import sparse

from src.data.preprocessing import UNSWPreprocessor

# notebook config
PROJECT_ROOT = Path.cwd().resolve().parents[1]

sys.path.insert(0, str(PROJECT_ROOT))

DATA_DIR = PROJECT_ROOT / "data" / "raw"

TRAIN_PATH = DATA_DIR / "UNSW_NB15_training-set.csv"
TEST_PATH = DATA_DIR / "UNSW_NB15_testing-set.csv"

print("Project root:", PROJECT_ROOT)
print("Train path:", TRAIN_PATH)
print("Test path:", TEST_PATH)

Project root: C:\Projects\Aeges-Q
Train path: C:\Projects\Aeges-Q\data\raw\UNSW_NB15_training-set.csv
Test path: C:\Projects\Aeges-Q\data\raw\UNSW_NB15_testing-set.csv
```

Preprocessing Strategy

Based on the exploratory data analysis, the UNSW-NB15 dataset requires a preprocessing pipeline that addresses categorical encoding, large differences in feature scale, highly skewed numerical distributions, and potential feature redundancy.

Rather than applying all transformations universally, multiple preprocessing variants will be constructed and evaluated according to their suitability for different downstream models.

```
# load dataset
train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)

print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)

Training shape: (82332, 45)
Testing shape: (175341, 45)
```

Preprocessing Variant A — Base Pipeline

This pipeline establishes the baseline preprocessing strategy for the UNSW-NB15 dataset. Numerical features are median-imputed, while categorical features are imputed using the most frequent value and one-hot encoded.

This representation serves as the reference point against which more specialized preprocessing strategies will be compared.

```
# preprocessor
preprocessor = UNSWPreprocessor()

X_train, y_train, attack_train = (
    preprocessor.split_features_and_target(train_df)
)

X_test, y_test, attack_test = (
    preprocessor.split_features_and_target(test_df)
)

preprocessor.identify_feature_types(X_train)

numerical_features = preprocessor.numerical_features
categorical_features = preprocessor.categorical_features

print(len(numerical_features))
print(len(categorical_features))

print(categorical_features)
```

```

39
3
['proto', 'service', 'state']

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)

print("\nTarget distribution:")
print(y_train.value_counts(normalize=True))

X_train: (82332, 42)
X_test: (175341, 42)

Target distribution:
label
1    0.5506
0    0.4494
Name: proportion, dtype: float64

```

Preprocessing Variant B — Standard Scaling

The EDA revealed large differences in numerical feature ranges. While tree-based models are generally insensitive to feature scaling, scaling is important for distance-based, margin-based, and quantum machine learning models.

This variant applies standardization to numerical features while preserving the existing categorical preprocessing strategy.

```

# scaled numerical pipeline
scaled_numeric_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median")
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)

# categorical pipeline
categorical_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="most_frequent")
        )
    ]
)

```

```

        ),
        (
            "encoder",
            OneHotEncoder(
                handle_unknown="ignore"
            )
        )
    ]
)

# scaled preprocessor
scaled_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            scaled_numeric_transformer,
            numerical_features
        ),
        (
            "categorical",
            categorical_transformer,
            categorical_features
        )
    ]
)

X_train_scaled = scaled_preprocessor.fit_transform(X_train)
X_test_scaled = scaled_preprocessor.transform(X_test)

print("Scaled train shape:", X_train_scaled.shape)
print("Scaled test shape:", X_test_scaled.shape)

Scaled train shape: (82332, 190)
Scaled test shape: (175341, 190)

scaled_feature_names = (
    scaled_preprocessor.get_feature_names_out()
)

numeric_feature_indices = [
    i
    for i, feature_name in enumerate(scaled_feature_names)
    if feature_name.startswith("numeric__")
]

X_train_scaled_numeric = (
    X_train_scaled[:, numeric_feature_indices]
)

```

```

numeric_means = np.asarray(
    X_train_scaled_numeric.mean(axis=0)
).ravel()

numeric_stds = np.sqrt(
    np.asarray(
        X_train_scaled_numeric.power(2).mean(axis=0)
    ).ravel()
    - numeric_means ** 2
)

print("Maximum absolute mean:", np.abs(numeric_means).max())

print("Minimum std:", numeric_stds.min())
print("Maximum std:", numeric_stds.max())

Maximum absolute mean: 1.5724755515983489e-13
Minimum std: 0.9999999999982245
Maximum std: 1.0000000000011098

```

Preprocessing Variant C — Robust Scaling

Several numerical features exhibit heavy skewness and extreme values. Unlike standard scaling, RobustScaler uses the median and interquartile range, making it less sensitive to the influence of outliers.

This variant is evaluated as an alternative numerical scaling strategy while retaining the same categorical preprocessing pipeline.

```

# numeric pipeline
robust_numeric_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median")
        ),
        (
            "scaler",
            RobustScaler()
        )
    ]
)

# transformer
robust_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",

```

```

        robust_numeric_transformer,
        numerical_features
    ),
    (
        "categorical",
        categorical_transformer,
        categorical_features
    )
]
)

# fit, train, transform and test
X_train_robust = robust_preprocessor.fit_transform(X_train)

X_test_robust = robust_preprocessor.transform(X_test)

print("Robust train shape:", X_train_robust.shape)
print("Robust test shape:", X_test_robust.shape)

Robust train shape: (82332, 190)
Robust test shape: (175341, 190)

# verification
robust_feature_names = robust_preprocessor.get_feature_names_out()

numeric_feature_indices = [
    i
    for i, feature_name in enumerate(robust_feature_names)
    if feature_name.startswith("numeric__")
]

X_train_robust_numeric = X_train_robust[:, numeric_feature_indices]
X_train_robust_numeric_dense = X_train_robust_numeric.toarray()

numeric_medians = np.median(
    X_train_robust_numeric_dense,
    axis=0
)

q1 = np.percentile(
    X_train_robust_numeric_dense,
    25,
    axis=0
)

q3 = np.percentile(
    X_train_robust_numeric_dense,
    75,
    axis=0
)

```

```

numeric_iqr = q3 - q1

print("Maximum absolute median:", np.abs(numeric_medians).max())

print("Minimum IQR:", numeric_iqr.min())
print("Maximum IQR:", numeric_iqr.max())

Maximum absolute median: 0.0
Minimum IQR: 0.0
Maximum IQR: 1.0

# finding 0 IQR features
zero_iqr_features = [
    numerical_features[i]
    for i, iqr in enumerate(numeric_iqr)
    if np.isclose(iqr, 0)
]

zero_iqr_features

['trans_depth',
 'response_body_len',
 'is_ftp_login',
 'ct_ftp_cmd',
 'ct_flw_http_mthd',
 'is_sm_ips_ports']

# non zero IQR features
nonzero_iqr = numeric_iqr[
    ~np.isclose(numeric_iqr, 0)
]

print("Zero-IQR features:", zero_iqr_features)
print("Number of zero-IQR features:", len(zero_iqr_features))

print("\nMinimum non-zero IQR:", nonzero_iqr.min())
print("Maximum non-zero IQR:", nonzero_iqr.max())

Zero-IQR features: ['trans_depth', 'response_body_len',
 'is_ftp_login', 'ct_ftp_cmd', 'ct_flw_http_mthd', 'is_sm_ips_ports']
Number of zero-IQR features: 6

Minimum non-zero IQR: 0.9999999999999999
Maximum non-zero IQR: 1.0

```

Preprocessing Variant D - Skew Aware Transformer

Preprocessing Variant D1 — Broad Log Transformation

The EDA identified several numerical features with heavily right-skewed distributions. Extreme skewness can cause a small number of large values to dominate the numerical range, potentially affecting models that are sensitive to feature distributions and scale.

This variant selectively applies a `log1p` transformation only to suitable non-negative numerical features with substantial positive skewness. The transformed features are then standardized, while the remaining numerical and categorical features follow their respective preprocessing pipelines.

The goal is to evaluate whether reducing distributional skewness improves the quality of the feature representation compared with the base and scaling-only variants.

```
# candidate features
skewness = X_train[numerical_features].skew()

skewed_features = skewness[
    skewness > 1
].index.tolist()

print("Number of heavily right-skewed features:",
      len(skewed_features))
print(skewed_features)

Number of heavily right-skewed features: 33
['dur', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'sload',
'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt', 'sjit', 'djit',
'tcprtt', 'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',
'response_body_len', 'ct_srv_src', 'ct_state_ttl', 'ct_dst_ltm',
'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm',
'is_ftp_login', 'ct_ftp_cmd', 'ct_flw_http_mthd', 'ct_src_ltm',
'ct_srv_dst', 'is_sm_ips_ports']

invalid_log_features = [
    feature
    for feature in skewed_features
    if X_train[feature].min() < 0
]

print("Features with negative values:", invalid_log_features)
```


Features with negative values: []

```
# log features
```

```
log_features = [  
    feature  
    for feature in skewed_features  
    if feature not in invalid_log_features  
]
```

```
remaining_numeric_features = [  
    feature  
    for feature in numerical_features  
    if feature not in log_features  
]
```

```
print("Log-transformed features:", len(log_features))  
print("Remaining numerical features:",  
len(remaining_numeric_features))
```

Log-transformed features: 33
Remaining numerical features: 6

```
# log transformed numerical features
```

```
log_numeric_transformer = Pipeline(  
    steps=[  
        (  
            "imputer",  
            SimpleImputer(strategy="median")  
        ),  
        (  
            "log_transform",  
            FunctionTransformer(np.log1p)  
        ),  
        (  
            "scaler",  
            StandardScaler()  
        )  
    ]  
)
```

```
# remaining numerical features
```

```
remaining_numeric_transformer = Pipeline(  
    steps=[  
        (  
            "imputer",  
            SimpleImputer(strategy="median")  
        ),  
        (  
            "scaler",  
            StandardScaler()  
        )  
    ]  
)
```

```

    )
]
)

# skew aware transformer
skew_preprocessor = ColumnTransformer(
    transformers=[
        (
            "log_numeric",
            log_numeric_transformer,
            log_features
        ),
        (
            "remaining_numeric",
            remaining_numeric_transformer,
            remaining_numeric_features
        ),
        (
            "categorical",
            categorical_transformer,
            categorical_features
        )
    ]
)

# fit and transform
X_train_skew = skew_preprocessor.fit_transform(X_train)

X_test_skew = skew_preprocessor.transform(X_test)

print("Skew-aware train shape:", X_train_skew.shape)
print("Skew-aware test shape:", X_test_skew.shape)

Skew-aware train shape: (82332, 190)
Skew-aware test shape: (175341, 190)

# verification : before vs after skewness
# Extract feature names
X_train_log_transformed = X_train_skew[:, :len(log_features)]

X_train_log_dense = X_train_log_transformed.toarray()

skew_comparison = pd.DataFrame({
    "feature": log_features,
    "skew_before": [
        X_train[feature].skew()
        for feature in log_features
    ],
    "skew_after": [
        pd.Series(X_train_log_dense[:, i]).skew()
        for i in range(len(log_features))
    ]
})

```

```

    ]
})

skew_comparison["absolute_skew_reduction"] = (
    skew_comparison["skew_before"].abs()
    - skew_comparison["skew_after"].abs()
)

skew_comparison.sort_values(
    by="absolute_skew_reduction",
    ascending=False
)

```

	feature	skew_before	skew_after
absolute_skew_reduction			
19	trans_depth	170.794394	2.989419
1.678050e+02			
20	response_body_len	74.635200	4.111906
7.052329e+01			
13	djit	60.562275	0.690386
5.987189e+01			
3	sbytes	53.778546	1.114123
5.266442e+01			
9	dloss	54.465649	1.827054
5.263860e+01			
4	dbytes	52.550387	0.248942
5.230144e+01			
8	sloss	52.465277	1.265456
5.119982e+01			
2	dpkts	49.304127	0.692811
4.861132e+01			
1	spkts	47.747777	1.075176
4.667260e+01			
11	dinpkt	23.013046	0.749781
2.226327e+01			
12	sjit	15.349380	0.242726
1.510665e+01			
6	sload	9.488393	-0.254374
9.234019e+00			
10	sinpkt	9.274082	0.962919
8.311163e+00			
29	ct_flw_http_mthd	12.557680	4.377478
8.180201e+00			
15	synack	13.809682	6.419753
7.389930e+00			
0	dur	9.523750	3.224474
6.299277e+00			
14	tcprrtt	9.060344	3.756920
5.303424e+00			
16	ackdat	9.628722	5.082881

4.545841e+00				
7	dload	4.753901	0.215175	
4.538725e+00				
5	rate	3.499493	-0.033330	
3.466163e+00				
18	dmean	3.106839	0.073006	
3.033833e+00				
17	smean	3.544111	1.909331	
1.634780e+00				
30	ct_src_ltm	2.437292	0.836432	
1.600861e+00				
23	ct_dst_ltm	2.667001	1.105902	
1.561100e+00				
21	ct_srv_src	1.856368	0.416678	
1.439690e+00				
31	ct_srv_dst	1.898853	0.489314	
1.409539e+00				
24	ct_src_dport_ltm	2.856818	1.482725	
1.374093e+00				
22	ct_state_ttl	1.522167	-0.352193	
1.169974e+00				
26	ct_dst_src_ltm	2.189041	1.071636	
1.117405e+00				
25	ct_dst_sport_ltm	2.506801	1.710249	7.965518e-
01				
28	ct_ftp_cmd	11.243182	10.977982	2.651991e-
01				
27	is_ftp_login	11.042308	10.929313	1.129948e-
01				
32	is_sm_ips_ports	9.321830	9.321830	1.776357e-
15				

Preprocessing Variant D2 — Selective Log Transformation

The broad log transformation experiment showed that `log1p` substantially reduced skewness for most heavily right-skewed numerical features. However, several sparse or indicator-like features showed negligible improvement after transformation.

This variant selectively applies `log1p` only to features for which the broad transformation demonstrated a meaningful reduction in skewness. Features with negligible improvement remain untransformed and are only scaled.

The objective is to determine whether a more targeted transformation strategy produces a better feature representation than applying the same transformation to all highly skewed features.

```
# exclude from log transformation features with skewness reduction less than 0.1
```

```
no_log_features = [  
    "is_ftp_login",  
    "ct_ftp_cmd",  
    "is_sm_ips_ports"  
]
```

```
# selective feature groups
```

```
selective_log_features = [  
    feature  
    for feature in log_features  
    if feature not in no_log_features  
]
```

```
selective_remaining_numeric_features = [  
    feature  
    for feature in numerical_features  
    if feature not in selective_log_features  
]
```

```
print(  
    "Features receiving log transformation:",  
    len(selective_log_features)  
)
```

```
print(  
    "Remaining numerical features:",  
    len(selective_remaining_numeric_features)  
)
```

```
print("\nLog-transformed features:")  
print(selective_log_features)
```

```
print("\nNon-log numerical features:")  
print(selective_remaining_numeric_features)
```

```
Features receiving log transformation: 30
```

```
Remaining numerical features: 9
```

```
Log-transformed features:
```

```
['dur', 'spkts', 'dpkts', 'sbytes', 'dbytes', 'rate', 'sload',  
'dload', 'sloss', 'dloss', 'sinpkt', 'dinpkt', 'sjit', 'djit',  
'tcprtt', 'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',  
'response_body_len', 'ct_srv_src', 'ct_state_ttl', 'ct_dst_ltm',  
'ct_src_dport_ltm', 'ct_dst_sport_ltm', 'ct_dst_src_ltm',  
'ct_flw_http_mthd', 'ct_src_ltm', 'ct_srv_dst']
```

```
Non-log numerical features:
```

```
['sttl', 'dttl', 'swin', 'stcpb', 'dtcpb', 'dwin', 'is_ftp_login',  
'ct_ftp_cmd', 'is_sm_ips_ports']
```

```
# log branch
```

```
selective_log_numeric_transformer = Pipeline(  
    steps=[  
        (  
            "imputer",  
            SimpleImputer(strategy="median")  
        ),  
        (  
            "log_transform",  
            FunctionTransformer(np.log1p)  
        ),  
        (  
            "scaler",  
            StandardScaler()  
        )  
    ]  
)
```

```
# remaining numeric branch
```

```
selective_remaining_numeric_transformer = Pipeline(  
    steps=[  
        (  
            "imputer",  
            SimpleImputer(strategy="median")  
        ),  
        (  
            "scaler",  
            StandardScaler()  
        )  
    ]  
)
```

```
# transformer
```

```
selective_skew_preprocessor = ColumnTransformer(  
    transformers=[  
        (  
            "log_numeric",  
            selective_log_numeric_transformer,  
            selective_log_features  
        ),  
        (  
            "remaining_numeric",  
            selective_remaining_numeric_transformer,  
            selective_remaining_numeric_features  
        ),  
        (  
            "categorical",
```

```

        categorical_transformer,
        categorical_features
    )
    ]
)

# fit and transform
X_train_selective_skew = (
    selective_skew_preprocessor.fit_transform(X_train)
)

X_test_selective_skew = (
    selective_skew_preprocessor.transform(X_test)
)

print(
    "Selective skew-aware train shape:",
    X_train_selective_skew.shape
)

print(
    "Selective skew-aware test shape:",
    X_test_selective_skew.shape
)

Selective skew-aware train shape: (82332, 190)
Selective skew-aware test shape: (175341, 190)

```

Preprocessing Variant E — Feature Redundancy Reduction

The EDA revealed strong correlations between several numerical features, indicating potential redundancy in the feature space. Highly correlated features may provide overlapping information, increasing dimensionality without necessarily contributing additional predictive value.

This variant investigates correlation-based feature reduction by systematically identifying highly correlated numerical features and applying a reproducible pruning strategy.

Rather than arbitrarily removing features, correlated feature groups will first be identified and analyzed before determining which features should be retained. The resulting reduced representation will then be evaluated against the previous preprocessing variants.

```

# correlation matrix for numerical features
correlation_matrix = X_train[numerical_features].corr()

```

```

correlation_matrix.shape
(39, 39)
# highly correlated feature pairs
upper_triangle = correlation_matrix.where(
    np.triu(
        np.ones(correlation_matrix.shape),
        k=1
    ).astype(bool)
)
correlation_threshold = 0.90
high_correlation_pairs = []
for column in upper_triangle.columns:
    correlated_features = upper_triangle.index[
        upper_triangle[column].abs() > correlation_threshold
    ].tolist()
    for feature in correlated_features:
        high_correlation_pairs.append(
            {
                "feature_1": feature,
                "feature_2": column,
                "correlation": correlation_matrix.loc[
                    feature,
                    column
                ]
            }
        )
high_correlation_df = pd.DataFrame(
    high_correlation_pairs
)
high_correlation_df = high_correlation_df.sort_values(
    by="correlation",
    key=lambda x: x.abs(),
    ascending=False
)
high_correlation_df

```

	feature_1	feature_2	correlation
5	dbytes	dloss	0.997109
3	sbytes	sloss	0.995027

11	is_ftp_login	ct_ftp_cmd	0.994341
4	dpkts	dloss	0.981506
14	ct_srv_src	ct_srv_dst	0.977849
1	dpkts	dbytes	0.976419
2	spkts	sloss	0.973644
0	spkts	sbytes	0.965750
8	ct_dst_ltm	ct_src_dport_ltm	0.960401
6	swin	dwin	0.960125
16	sinpkt	is_sm_ips_ports	0.944506
15	ct_dst_src_ltm	ct_srv_dst	0.941047
7	tcprrt	synack	0.939473
10	ct_srv_src	ct_dst_src_ltm	0.933795
13	ct_src_dport_ltm	ct_src_ltm	0.933172
12	ct_dst_ltm	ct_src_ltm	0.932252
9	ct_src_dport_ltm	ct_dst_sport_ltm	0.911637

```
# build correlation graph
```

```
correlation_graph = nx.Graph()
```

```
for _, row in high_correlation_df.iterrows():
```

```
    correlation_graph.add_edge(
        row["feature_1"],
        row["feature_2"],
        weight=abs(row["correlation"])
    )
```

```
# feature groups based on correlation
```

```
correlated_feature_groups = list(
    nx.connected_components(correlation_graph)
)
```

```
for i, group in enumerate(correlated_feature_groups, start=1):
```

```
    print(f"Group {i}:")
    print(sorted(group))
    print()
```

```
Group 1:
['dbytes', 'dloss', 'dpkts']
```

```
Group 2:
['sbytes', 'sloss', 'spkts']
```

```
Group 3:
['ct_ftp_cmd', 'is_ftp_login']
```

```
Group 4:
['ct_dst_src_ltm', 'ct_srv_dst', 'ct_srv_src']
```

```
Group 5:
```

```
['ct_dst_ltm', 'ct_dst_sport_ltm', 'ct_src_dport_ltm', 'ct_src_ltm']
```

Group 6:

```
['dwin', 'swin']
```

Group 7:

```
['is_sm_ips_ports', 'sinpkt']
```

Group 8:

```
['synack', 'tcprtt']
```

Feature Relevance Within Correlated Groups

Highly correlated features may still differ in their relationship with the target variable. To avoid arbitrarily selecting a representative feature from each correlated group, Mutual Information is used to estimate the relationship between each numerical feature and the binary attack label.

Within each correlated group, the feature with the highest Mutual Information score will be retained as the representative feature, while the remaining features will be considered redundant for this preprocessing variant.

```
# calculate mutual information for each feature with respect to the target variable
```

```
X_numeric = X_train[numerical_features].copy()
```

```
X_numeric = X_numeric.fillna(  
    X_numeric.median()  
)
```

```
mi_scores = mutual_info_classif(  
    X_numeric,  
    y_train,  
    random_state=42  
)
```

```
mi_df = pd.DataFrame({  
    "feature": numerical_features,  
    "mutual_information": mi_scores  
})
```

```
mi_df = mi_df.sort_values(  
    by="mutual_information",  
    ascending=False  
)
```

```
mi_df
```

	feature	mutual_information
3	sbytes	0.460296

23	smean	0.357387
8	sload	0.345558
4	dbytes	0.294109
28	ct_state_ttl	0.280379
5	rate	0.263027
0	dur	0.253707
24	dmean	0.249968
7	dttl	0.242774
13	dinpkt	0.238735
6	sttl	0.218773
9	dload	0.212826
21	synack	0.207825
2	dpkts	0.207486
20	tcprtt	0.206408
12	sinpkt	0.198707
31	ct_dst_sport_ltm	0.187999
22	ackdat	0.186647
14	sjit	0.161172
1	spkts	0.152714
15	djit	0.148359
11	dloss	0.117450
30	ct_src_dport_ltm	0.116976
37	ct_srv_dst	0.115466
10	sloss	0.114317
16	swin	0.095099
17	stcpb	0.092392
27	ct_srv_src	0.090650
18	dtcpb	0.083768
19	dwin	0.076078
32	ct_dst_src_ltm	0.069721
29	ct_dst_ltm	0.061561
36	ct_src_ltm	0.058630
26	response_body_len	0.040204
38	is_sm_ips_ports	0.007215
35	ct_flw_http_mthd	0.005561
25	trans_depth	0.004428
33	is_ftp_login	0.001134
34	ct_ftp_cmd	0.000000

```

for i, group in enumerate(
    correlated_feature_groups,
    start=1
):

    group_mi = mi_df[
        mi_df["feature"].isin(group)
    ].sort_values(
        by="mutual_information",
        ascending=False
    )

```

```
print(f"\nGroup {i}")
print(group_mi.to_string(index=False))
```

Group 1

feature	mutual_information
dbytes	0.294109
dpkts	0.207486
dloss	0.117450

Group 2

feature	mutual_information
sbytes	0.460296
spkts	0.152714
sloss	0.114317

Group 3

feature	mutual_information
is_ftp_login	0.001134
ct_ftp_cmd	0.000000

Group 4

feature	mutual_information
ct_srv_dst	0.115466
ct_srv_src	0.090650
ct_dst_src_ltm	0.069721

Group 5

feature	mutual_information
ct_dst_sport_ltm	0.187999
ct_src_dport_ltm	0.116976
ct_dst_ltm	0.061561
ct_src_ltm	0.058630

Group 6

feature	mutual_information
swin	0.095099
dwin	0.076078

Group 7

feature	mutual_information
sinpkt	0.198707
is_sm_ips_ports	0.007215

Group 8

feature	mutual_information
synack	0.207825
tcprtt	0.206408

```

# automatically select representatives
selected_correlated_features = []

for group in correlated_feature_groups:
    group_mi = mi_df[
        mi_df["feature"].isin(group)
    ]

    best_feature = group_mi.loc[
        group_mi["mutual_information"].idxmax(),
        "feature"
    ]

    selected_correlated_features.append(best_feature)

selected_correlated_features

['dbytes',
'sbytes',
'is_ftp_login',
'ct_srv_dst',
'ct_dst_sport_ltm',
'swin',
'sinpkt',
'synack']

# determine features to drop based on correlation and mutual
information
features_to_drop = []

for group in correlated_feature_groups:
    for feature in group:
        if feature not in selected_correlated_features:
            features_to_drop.append(feature)

features_to_drop = sorted(features_to_drop)

print("Number of numerical features:", len(numerical_features))
print("Features retained from correlated groups:",
len(selected_correlated_features))
print("Features to drop:", len(features_to_drop))

print("\nFeatures to drop:")
print(features_to_drop)

Number of numerical features: 39
Features retained from correlated groups: 8
Features to drop: 13

```

```

Features to drop:
['ct_dst_ltm', 'ct_dst_src_ltm', 'ct_ftp_cmd', 'ct_src_dport_ltm',
'ct_src_ltm', 'ct_srv_src', 'dloss', 'dpkts', 'dwin',
'is_sm_ips_ports', 'sloss', 'spkts', 'tcprrt']

# create reduced numerical features list
reduced_numerical_features = [
    feature
    for feature in numerical_features
    if feature not in features_to_drop
]

print("Reduced numerical feature count:")
print(len(reduced_numerical_features))

print("\nRemaining numerical features:")
print(reduced_numerical_features)

Reduced numerical feature count:
26

Remaining numerical features:
['dur', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload', 'dload',
'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'stcpb', 'dtcpb',
'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',
'response_body_len', 'ct_state_ttl', 'ct_dst_sport_ltm',
'is_ftp_login', 'ct_flw_http_mthd', 'ct_srv_dst']

# numerical pipeline
reduced_numeric_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median")
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)

# preprocessor
redundancy_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            reduced_numeric_transformer,
            reduced_numerical_features
        ),
    ],

```

```

        (
            "categorical",
            categorical_transformer,
            categorical_features
        )
    ]
)

# fit on training data and transform both training and testing data
X_train_reduced = redundancy_preprocessor.fit_transform(X_train)

X_test_reduced = redundancy_preprocessor.transform(X_test)

print(
    "Reduced train shape:",
    X_train_reduced.shape
)

print(
    "Reduced test shape:",
    X_test_reduced.shape
)

Reduced train shape: (82332, 177)
Reduced test shape: (175341, 177)

if sparse.issparse(X_train_reduced):
    train_has_nan = np.isnan(X_train_reduced.data).any()
    test_has_nan = np.isnan(X_test_reduced.data).any()
else:
    train_has_nan = np.isnan(X_train_reduced).any()
    test_has_nan = np.isnan(X_test_reduced).any()

print(
    "Train and test feature spaces match:",
    X_train_reduced.shape[1] == X_test_reduced.shape[1]
)

print("NaNs in train:", train_has_nan)
print("NaNs in test:", test_has_nan)

Train and test feature spaces match: True
NaNs in train: False
NaNs in test: False

```

Summary of Preprocessing Experiments

The preprocessing stage translated the findings from the exploratory data analysis into a set of reproducible candidate pipelines.

The objective was not to assume that a single transformation strategy would be optimal for all downstream models. Instead, multiple preprocessing variants were constructed to address different characteristics of the UNSW-NB15 feature space, including categorical variables, differences in numerical scale, extreme skewness, outliers, and feature redundancy.

All transformers were fitted exclusively on the training data and subsequently applied to the testing data to prevent information leakage.

Base Feature Representation

The original model input contains **42 features**:

- **39 numerical features**
- **3 categorical features:** `proto`, `service`, and `state`

The identifier column (`id`), binary target (`label`), and attack-category metadata (`attack_cat`) were excluded from the model input.

The categorical features were handled using:

- Most-frequent value imputation
- One-hot encoding with unknown-category handling

After encoding, the original 42-feature representation expands to a processed feature space of **190 features**.

The training and testing sets produced identical feature spaces, confirming that the preprocessing pipeline handles the categorical representation consistently.

Preprocessing Variant A — Base Pipeline

The base pipeline provides the minimum required preprocessing:

Numerical features

- Median imputation

Categorical features

- Most-frequent value imputation
- One-hot encoding

This variant serves as the reference representation against which the more specialized preprocessing strategies will be evaluated.

Preprocessing Variant B — Standard Scaling

The second variant applies standardization to the numerical features.

Numerical features

- Median imputation
- Standard scaling

Categorical features

- Most-frequent value imputation
- One-hot encoding

This representation is particularly relevant for models that are sensitive to feature magnitude, such as distance-based and margin-based classifiers.

Verification confirmed that the transformed numerical features have means approximately equal to zero and standard deviations approximately equal to one.

Preprocessing Variant C — Robust Scaling

Because several numerical features exhibit extreme values and heavily skewed distributions, a robust scaling variant was also constructed.

Numerical features

- Median imputation
- Robust scaling

Categorical features

- Most-frequent value imputation
- One-hot encoding

Robust scaling uses statistics based on the median and interquartile range, reducing the influence of extreme observations compared with standard scaling.

Several features have zero interquartile range because of their highly discrete or sparse distributions. These features were retained rather than arbitrarily removed, while the scaling behaviour was explicitly verified.

Preprocessing Variant D1 — Broad Log Transformation

EDA identified **33 highly right-skewed numerical features** with absolute skewness greater than 1.

The first skew-aware strategy applied a `log1p` transformation to all non-negative highly skewed features before standardization.

Highly skewed numerical features

- Median imputation
- `log1p` transformation
- Standard scaling

Remaining numerical features

- Median imputation
- Standard scaling

Categorical features

- Most-frequent value imputation
- One-hot encoding

The transformation substantially reduced skewness for many features, although some highly discrete or sparse variables showed little improvement.

Preprocessing Variant D2 — Selective Log Transformation

The broad transformation experiment showed that applying `log1p` to every highly skewed feature was not equally beneficial.

Therefore, a more selective strategy was created.

Only features for which the log transformation meaningfully improved the distribution were transformed.

Selected skewed features

- Median imputation
- `log1p` transformation
- Standard scaling

Remaining numerical features

- Median imputation
- Standard scaling

Categorical features

- Most-frequent value imputation
- One-hot encoding

This approach retained the benefits of skew reduction while avoiding unnecessary transformations for features whose distributions were not meaningfully improved by the logarithmic transformation.

A total of **30 numerical features** received log transformation, while **9 numerical features** remained untransformed before scaling.

Preprocessing Variant E — Correlation-Based Redundancy Reduction

Correlation analysis identified **8 groups of strongly correlated numerical features** using a high-correlation threshold.

Rather than arbitrarily selecting one feature from each group, Mutual Information with the binary target was used to select the representative feature with the strongest individual relationship to the attack label.

The selected representative features were:

- dbytes
- sbytes
- is_ftp_login
- ct_srv_dst
- ct_dst_sport_ltm
- swin
- sinpkt
- synack

This resulted in **13 correlated numerical features being removed**.

The numerical feature space was therefore reduced from:

39 numerical features → 26 numerical features

The resulting preprocessing strategy consists of:

Reduced numerical features

- Median imputation
- Standard scaling

Categorical features

- Most-frequent value imputation
- One-hot encoding

After one-hot encoding, the processed dimensionality was reduced from:

190 features → 177 features

The transformed training and testing sets have matching feature spaces and contain no missing values.

Final Preprocessing Strategy

At this stage, no single preprocessing variant is assumed to be universally optimal.

The following candidate representations will be carried forward for empirical comparison:

Variant	Numerical Processing	Categorical Processing	Processed Feature Space
A	Median imputation	Imputation + One-hot encoding	190
B	Median imputation + StandardScaler	Imputation + One-hot encoding	190
C	Median imputation + RobustScaler	Imputation + One-hot encoding	190
D1	Broad <code>log1p</code> + StandardScaler	Imputation + One-hot encoding	190
D2	Selective <code>log1p</code> + StandardScaler	Imputation + One-hot encoding	190
E	Redundancy reduction + StandardScaler	Imputation + One-hot encoding	177

The next stage of the project will benchmark these preprocessing variants using the same downstream classical machine-learning models and evaluation protocol.

The purpose of this comparison is to empirically determine whether scaling, robust scaling, skew-aware transformation, or correlation-based feature reduction provides the most useful representation for intrusion detection.

The strongest preprocessing strategy will then be selected and carried forward into subsequent feature-reduction and constrained QML experiments.